

thread in the team. At the end of the parallel construct, all threads synchronise and get terminated. Only the master thread continues the further execution. Figure 1 depicts the fork-join model of OpenMP.

Within a single program, you can mark any number of parallel constructs.



Note: Team of Thread = Master + Workers.

When any thread team encounters a work sharing construct, the work inside the construct is offloaded and distributed among its members, and gets executed cooperatively, instead of the whole work being executed by every thread, multiple times. Each thread just has its portion of work, which it executes synchronously.

When can you consider OpenMP?

- When data parallelism is required. According to Wikipedia, data parallelism (also called loop-parallelism) is achieved when each processor performs the same task on different pieces of distributed data.
- When a block of code is required to be executed in-parallel.
- Importantly, when you don't want to bring modification to your existing application design, and yet want to achieve parallelism. All you need to do is to carefully analyse your code and insert appropriate compiler directives, and compile your code with the compiler that supports OpenMP. With compilers that don't support OpenMP, just ignore these directives.

How is OpenMP used?

Well, as already mentioned, an OpenMP program consists of: compiler directives, library routines and environment variables. OpenMP directives for C/C++ are specified with the `pragma` pre-processing directive, and each directive starts with `#pragma omp`.

You need to first specify how many threads you want to create for your parallel regions. To do so, follow the steps given below:

- At run-time with `omp_set_num_threads`:

```
#include <omp.h>
omp_set_num_threads(int num_threads)
```
- Or, via environment variable `OMP_NUM_THREADS`:

```
export OMP_NUM_THREADS=<specify the number of threads you want>
```

Function prototypes and types are in the `omp.h` file.

Finally, you can compile your OpenMP code with the `-fopenmp` switch:

```
$ gcc <your C file> -o <output file> -fopenmp
```

The next section deals with useful OpenMP directives

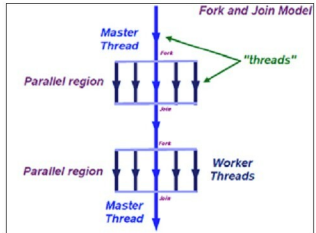


Figure 1: OpenMP's fork-join model

for *parallel* processing.

Some basic OpenMP constructs

The fundamental construct of OpenMP is *parallel*. This construct starts *parallel* execution. Without this construct, your code will remain sequential:

```
/* Sequential Code here..(Master Thread) */
#pragma omp parallel
{ /* A team of N Threads starts here */
  /* Team of Threads = Master + Worker Threads */
  /* Parallel computing here... */
}
/* Master Thread resumes back here */
```

Just a *parallel* region alone doesn't specify how the work will be shared. And thus all threads will end up redundantly executing all the work—that is, no work sharing and, therefore, no actual speed gain. Compile the example given next with GCC and the `-fopenmp` switch, and run it to see the output.

```
#include <omp.h>
#include <stdio.h>

int main (int argc, char *argv[])
{
  int nthreads, tid;

  omp_set_num_threads(4);
  /* Just fork a team of threads */
  #pragma omp parallel
  {
    /* Obtain thread number */
    tid = omp_get_thread_num();
    printf("Hello World from thread = %d\n", tid);

    /* Only master thread does this */
    if (tid == 0)
```